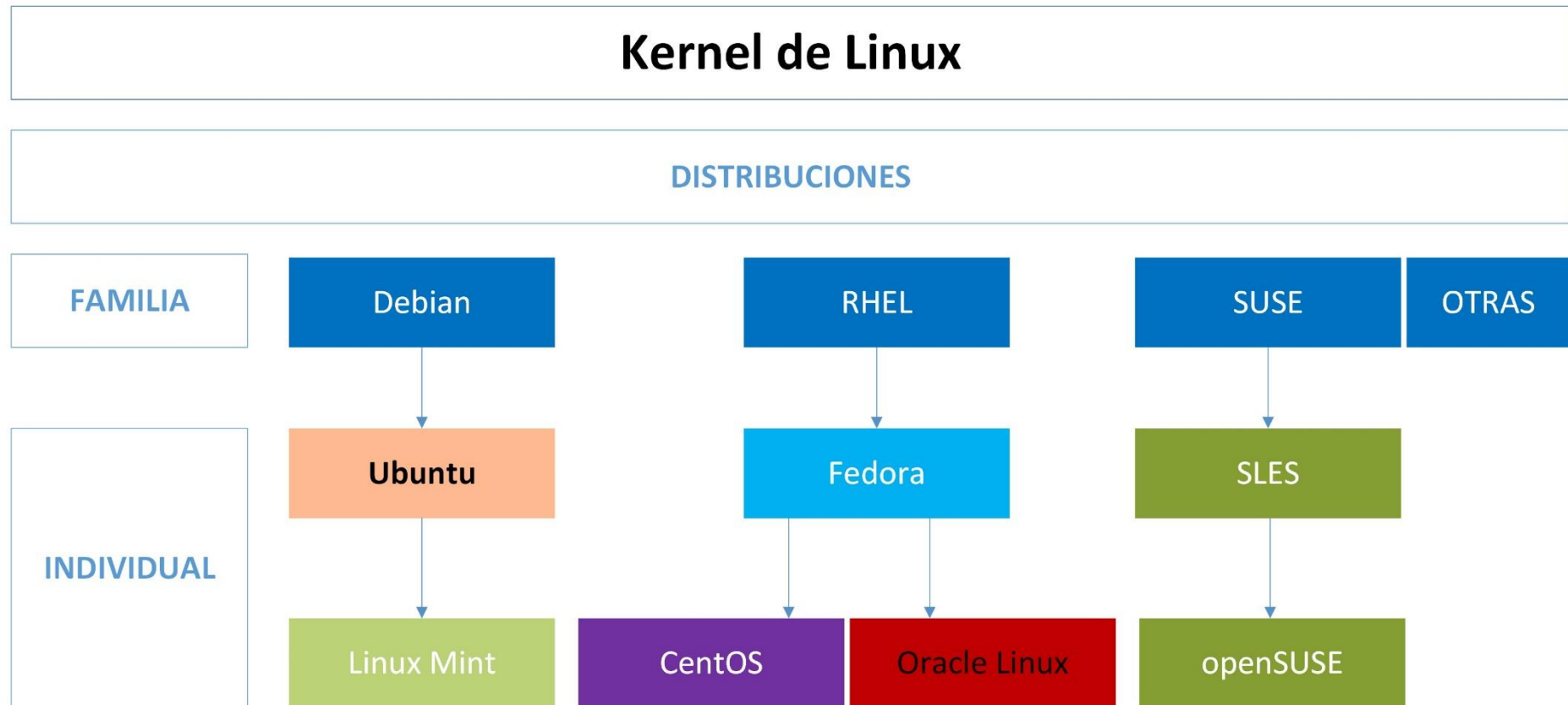


23-24: Instalación de Docker sobre kali-linux

Instalación de imágenes CentOS y CentOS 7

Hello World sobre la imagen de CentOS 7

Distribuciones de Linux.

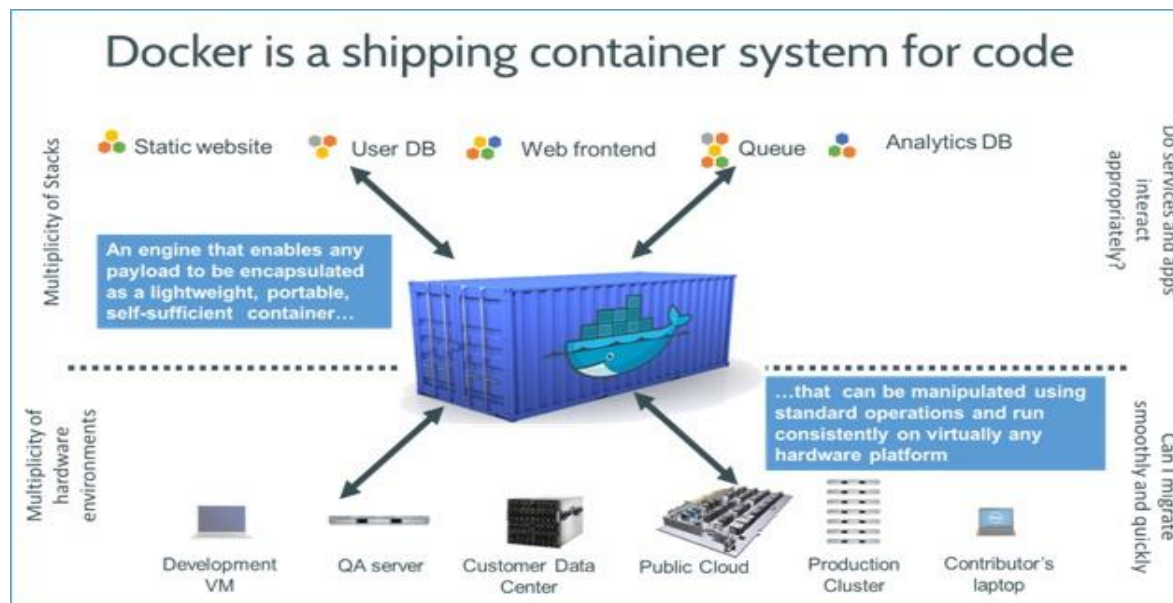


Docker.

Introducción.

La virtualización a nivel de sistema operativo nos permite ejecutar varias instancias aisladas de espacio de usuario en paralelo. Estas instancias de espacio de usuario incluyen el código fuente de la aplicación, las bibliotecas necesarias y el tiempo de ejecución necesario para ejecutar la aplicación sin dependencias externas. Estas instancias de espacio de usuario se conocen como **contenedores**. Resuelven los desafíos de conflicto de dependencias, bibliotecas y tiempos de ejecución a los que los desarrolladores se han estado enfrentando en el pasado.

Los desarrolladores prefieren no preocuparse por los aspectos de portabilidad y coherencia de las aplicaciones y esperan que la aplicación simplemente funcione, independientemente de la plataforma subyacente. Mediante el uso de una tecnología de **contenedor como Docker**, la aplicación y todas sus dependencias se pueden agrupar en un contenedor (caja).



Fuente: [What are containers? | IBM](#)

La caja (contenedor) se puede enviar (transportar) a diferentes plataformas y se ejecutará de forma idéntica en cada una de ellas.

El concepto de transporte es similar a los contenedores marítimos.

Fuente: Google Images.



En el mundo del contenedor, esta caja que contiene nuestro código fuente de la aplicación y todas sus dependencias y bibliotecas se conoce como una **imagen**.

Una **instancia en ejecución** de esta caja se conoce como **contenedor**. Podemos ejecutar varios contenedores desde la misma imagen.

Una imagen contiene la aplicación, sus dependencias y las bibliotecas de espacio de usuario. Las bibliotecas de espacio de usuario como glibc permiten cambiar del espacio de usuario al espacio del núcleo. Una imagen no contiene ningún componente del espacio del núcleo.

Cuando se crea un contenedor a partir de una imagen, **se ejecuta como un proceso en el kernel del host**. El trabajo del kernel del host es aislar el proceso de contenedor y proporcionar recursos a cada contenedor.

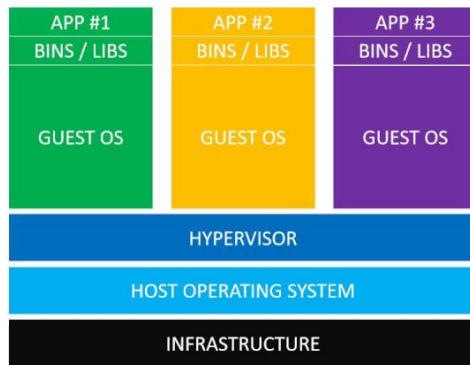
- **Namespaces** (Espacio de nombres)

Un espacio de nombres encapsula un recurso del sistema global determinado como la red, los identificadores de proceso en una abstracción, que hace que parezca a los procesos dentro del espacio de nombres que tienen su propia instancia aislada del recurso global

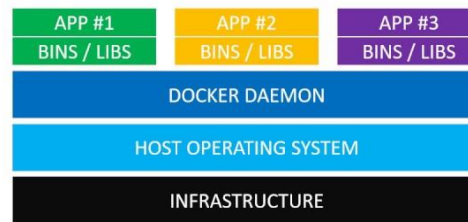
En los últimos años, se ha experimentado un rápido crecimiento en el interés y la adopción de tecnologías de contenedores. La mayoría de los proveedores de nube y proveedores de TI ofrecen soporte para contenedores. Para asegurarse de que no hay bloqueo de proveedores y ninguna inclinación hacia una empresa o proyecto en particular, las empresas de TI se unieron y formaron una estructura de gobierno abierto, llamada The Open Container Initiative (**OCI**). <https://opencontainers.org/>

- **Containerd**

Containerd es un motor de ejecución de contenedor compatible con OCI con énfasis en la simplicidad, robustez y portabilidad. Se ejecuta como un demonio y administra todo el ciclo de vida de los contenedores. Está disponible en Linux y Windows. Docker, que es una plataforma de contenedor, usa **containerd** como motor de ejecución de contenedor para administrar contenedores.



Virtual Machines



Docker Containers

Fuente: Google Images.

A diferencia de las máquinas virtuales, **los contenedores se ejecutan directamente como procesos en el sistema operativo host**. No hay direccionamiento indirecto como vemos en las máquinas virtuales, lo que ayuda a los contenedores a lograr un rendimiento casi nativo. Además, como los contenedores tienen una huella muy ligera, resulta más fácil empaquetar un mayor número de contenedores que las máquinas virtuales en la misma máquina física.

23-24: La práctica de Docker la aplicamos sobre kali-linux

Instalación de Docker

- Instalar docker en la máquina `sudo apt install docker.io`

```
ma@LAPTOP-0VBOP51T: ~  
ma@LAPTOP-0VBOP51T:~$ sudo apt install docker.io  
Reading package lists... Done  
Building dependency tree  
Reading state information... Done  
The following additional packages will be installed:  
  bridge-utils cgroupfs-mount containerd dns-root-data dnsmasq-base libidn11 pigz runc ubuntu-fan  
Suggested packages:  
  ifupdown aufs-tools debootstrap docker-doc rinse zfs-fuse | zfsutils  
The following NEW packages will be installed:  
  bridge-utils cgroupfs-mount containerd dns-root-data dnsmasq-base docker.io libidn11 pigz runc ub  
0 upgraded, 10 newly installed, 0 to remove and 81 not upgraded.  
Need to get 69.7 MB of archives.  
After this operation, 334 MB of additional disk space will be used.  
Do you want to continue? [Y/n] y  
Get:1 http://archive.ubuntu.com/ubuntu focal/universe amd64 pigz amd64 2.4-1 [57.4 kB]  
Get:2 http://archive.ubuntu.com/ubuntu focal/main amd64 bridge-utils amd64 1.6-2ubuntu1 [30.5 kB]  
Get:3 http://archive.ubuntu.com/ubuntu focal/universe amd64 cgroupfs-mount all 1.4 [6320 B]  
Get:4 http://archive.ubuntu.com/ubuntu focal/main amd64 runc amd64 1.0.0~rc10-0ubuntu1 [2549 kB]  
Get:5 http://archive.ubuntu.com/ubuntu focal/main amd64 containerd amd64 1.3.3-0ubuntu2 [27.8 MB]  
Get:6 http://archive.ubuntu.com/ubuntu focal/main amd64 dns-root-data all 2019052802 [5300 B]  
Get:7 http://archive.ubuntu.com/ubuntu focal/main amd64 libidn11 amd64 1.33-2.2ubuntu2 [46.2 kB]  
Get:8 http://archive.ubuntu.com/ubuntu focal/main amd64 dnsmasq-base amd64 2.80-1.1ubuntu1 [314 kB]  
Get:9 http://archive.ubuntu.com/ubuntu focal-updates/universe amd64 docker.io amd64 19.03.8-0ubuntu  
Get:10 http://archive.ubuntu.com/ubuntu focal/main amd64 ubuntu-fan all 0.12.13 [34.5 kB]  
Fetched 69.7 MB in 57s (1216 kB/s)  
Preconfiguring packages ...  
Selecting previously unselected package pigz.  
(Reading database ... 37478 files and directories currently installed.)  
Preparing to unpack .../0-pigz_2.4-1_amd64.deb
```

- Comprobar la versión instalada con el comando

`docker --version`

ma@LAPTOP-0VBOP51T: ~

```
ma@LAPTOP-0VBOP51T:~$ docker --version
Docker version 19.03.8, build afacb8b7f0
ma@LAPTOP-0VBOP51T:~$
```

ma@LAPTOP-0VBOP51T: ~/SS00/y21/practica1

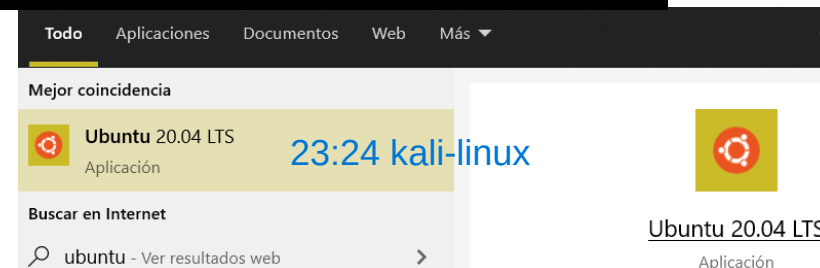
```
ma@LAPTOP-0VBOP51T:~/SS00/y21/practica1$ docker --version
Docker version 20.10.7, build 20.10.7-0ubuntu1~20.04.1
ma@LAPTOP-0VBOP51T:~/SS00/y21/practica1$
```

- Ejecución
- 1. Docker Daemon.

Primero, debemos tener el Docker daemon corriendo `sudo dockerd`

```
ma@LAPTOP-0VBOP51T: ~/SSOO/y21/practica1
ma@LAPTOP-0VBOP51T:~/SSOO/y21/practica1$ sudo dockerd
[sudo] password for ma:
INFO[2021-09-26T13:59:41.246007600+02:00] Starting up
INFO[2021-09-26T13:59:41.252173000+02:00] libcontainerd: started new containerd process pid=116
INFO[2021-09-26T13:59:41.252234700+02:00] parsed scheme: "unix" module=grpc
INFO[2021-09-26T13:59:41.252264600+02:00] scheme "unix" not registered, fallback to default scheme module=grpc
INFO[2021-09-26T13:59:41.252299200+02:00] ccResolverWrapper: sending update to cc: {[{unix:///var/run/docker/cont
ock <nil> 0 <nil>}] <nil> <nil>} module=grpc
INFO[2021-09-26T13:59:41.252367300+02:00] ClientConn switching balancer to "pick_first" module=grpc
WARN[0000] deprecated version : `1`, please switch to version `2`
INFO[2021-09-26T13:59:41.411217400+02:00] starting containerd revision= version="1.5.2-6
INFO[2021-09-26T13:59:41.431517700+02:00] loading plugin "io.containerd.content.v1.content"... type=io.containerd
INFO[2021-09-26T13:59:41.433175600+02:00] loading plugin "io.containerd.snapshotter.v1.aufs"... type=io.containerd
INFO[2021-09-26T13:59:41.445720100+02:00] skip loading plugin "io.containerd.snapshotter.v1.aufs"... error="aufs
modprobe aufs failed: exit status 1 \"modprobe: FATAL: Module aufs not found in directory /lib/modules/4.19.128-mi
h\"): skip plugin" type=io.containerd.snapshotter.v1
INFO[2021-09-26T13:59:41.445802200+02:00] loading plugin "io.containerd.snapshotter.v1.btrfs"... type=io.containe
INFO[2021-09-26T13:59:41.446046000+02:00] skip loading plugin "io.containerd.snapshotter.v1.btrfs"... error="path
ontainerd/daemon/io.containerd.snapshotter.v1.btrfs (ext4) must be a btrfs filesystem to be used with the btrfs sr
ugin" type=io.containerd.snapshotter.v1
INFO[2021-09-26T13:59:41.446103800+02:00] loading plugin "io.containerd.snapshotter.v1.devmapper"... type=io.cont
.v1
```

Ésta será la Terminal1, abrimos otra...



23-24. Instalación de dos imágenes, CentOS última versión y CentOS 7

Instalación de la imagen de CentOS mediante otra terminal.

ma@LAPTOP-0VBOP51T: ~/SS00/y21/practica1/contenedor

```
ma@LAPTOP-0VBOP51T:~/SS00/y21/practica1/contenedor$ sudo docker pull centos
[sudo] password for ma:
Using default tag: latest
latest: Pulling from library/centos
a1d0c7532777: Downloading [=====>] 38.62MB/83.52MB
```

sudo docker pull centos 23-24: sudo docker pull centos:7

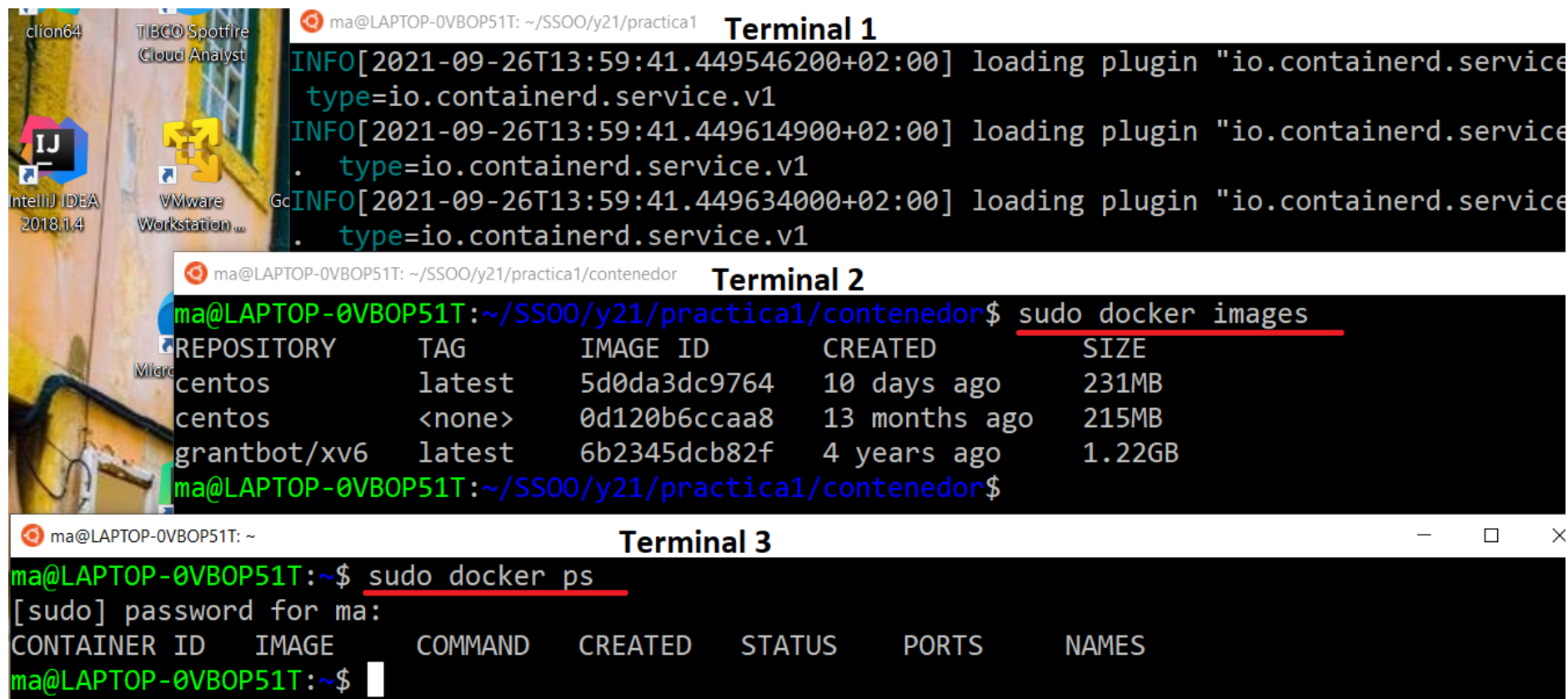
ma@LAPTOP-0VBOP51T: ~/SS00/y21/practica1/contenedor

```
ma@LAPTOP-0VBOP51T:~/SS00/y21/practica1/contenedor$ sudo docker pull centos
[sudo] password for ma:
Using default tag: latest
latest: Pulling from library/centos
a1d0c7532777: Pull complete
Digest: sha256:a27fd8080b517143cbbbab9dfb7c8571c40d67d534bbdee55bd6c473f432b177
Status: Downloaded newer image for centos:latest
docker.io/library/centos:latest
ma@LAPTOP-0VBOP51T:~/SS00/y21/practica1/contenedor$
```

Comprobación del registro de imagen.

Después de la instalación de la imagen, comprobamos su registro.

sudo docker images y los procesos **sudo docker ps**



The screenshot shows a Linux desktop environment with three terminal windows. The desktop background is a yellow and orange abstract image. Icons for 'clion64', 'TIBCO Spotfire Cloud Analyst', 'IntelliJ IDEA 2018.1.4', and 'VMware Workstation' are visible. The terminal windows are titled 'Terminal 1', 'Terminal 2', and 'Terminal 3'.

Terminal 1 shows the output of the Docker daemon startup:

```
ma@LAPTOP-0VBOP51T: ~/SSOO/y21/practica1
INFO[2021-09-26T13:59:41.449546200+02:00] loading plugin "io.containerd.service
type=io.containerd.service.v1
INFO[2021-09-26T13:59:41.449614900+02:00] loading plugin "io.containerd.service
. type=io.containerd.service.v1
INFO[2021-09-26T13:59:41.449634000+02:00] loading plugin "io.containerd.service
. type=io.containerd.service.v1
```

Terminal 2 shows the output of the `sudo docker images` command:

```
ma@LAPTOP-0VBOP51T: ~/SSOO/y21/practica1/contenedor$ sudo docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
centos	latest	5d0da3dc9764	10 days ago	231MB
centos	<none>	0d120b6ccaa8	13 months ago	215MB
grantbot/xv6	latest	6b2345dcb82f	4 years ago	1.22GB

Terminal 3 shows the output of the `sudo docker ps` command:

```
ma@LAPTOP-0VBOP51T: ~$ sudo docker ps
[sudo] password for ma:
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
ma@LAPTOP-0VBOP51T: ~$
```

23-24: sudo docker run --rm -ti centos:7 /bin/bash

Arrancamos una sesión bash en CentOS

`sudo docker run --rm -ti centos /bin/bash`

```
@a8266c6267cd:/
ma@LAPTOP-0VBOP51T:~/SS00/y21/practical/contenedor$ sudo docker images
REPOSITORY      TAG         IMAGE ID      CREATED       SIZE
centos           latest      5d0da3dc9764  10 days ago   231MB
centos           <none>      0d120b6ccaa8  13 months ago 215MB
grantbot/xv6     latest      6b2345dcb82f  4 years ago   1.22GB
ma@LAPTOP-0VBOP51T:~/SS00/y21/practical/contenedor$ sudo docker run --rm -ti centos /bin/bash
[root@a8266c6267cd /]# cat /etc/os-release
NAME="CentOS Linux"
VERSION="8"
ID="centos"
ID_LIKE="rhel fedora"
VERSION_ID="8"
PLATFORM_ID="platform:el8"
PRETTY_NAME="CentOS Linux 8"
ANSI_COLOR="0;31"
CPE_NAME="cpe:/o:centos:centos:8"
HOME_URL="https://centos.org/"
BUG_REPORT_URL="https://bugs.centos.org/"
CENTOS_MANTISBT_PROJECT="CentOS-8"
CENTOS_MANTISBT_PROJECT_VERSION="8"
[root@a8266c6267cd /]#
```

Ahora podemos comprobar que el contenedor está en ejecución.

```
ma@LAPTOP-0VBOP51T: ~
ma@LAPTOP-0VBOP51T:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
15ca772eae9b   centos    "/bin/bash"             8 minutes ago Up 8 minutes   hungry_wilbur
ma@LAPTOP-0VBOP51T:~$
```

23-24 sobre la sesión de CentOS 7

En la sesión de CentOS instalamos gcc y nano.

@15ca772eae9b:/

```
ma@LAPTOP-0VBOP51T:~/SS00/y21/practica1/contenedor$ sudo docker run --rm -ti centos /bin/bash
[root@15ca772eae9b /]# yum install gcc
Failed to set locale, defaulting to C.UTF-8
CentOS Linux 8 - AppStream          18% [=====] 235 kB/s | 1.7 MB
```

@15ca772eae9b:/

```
[root@15ca772eae9b /]# yum install nano
Failed to set locale, defaulting to C.UTF-8
Last metadata expiration check: 0:01:45 ago on Sun Sep 26 12:39:44 2021.
Dependencies resolved.
```

Package	Architecture	Version
Installing:		
nano	x86_64	2.9.8-1.el8

Transaction Summary

Install 1 Package

Total download size: 581 k
Installed size: 2.2 M
Is this ok [y/N]: y
Downloading Packages:
nano-2.9.8-1.el8.x86_64.rpm

Total
Running transaction check

```
#include <stdio.h>

int main(){
    printf("Hello World!, we are INSO SIOP from CentOS in a docker container");
    return 0;
}
```

Installed:

nano-2.9.8-1.el8.x86_64

Complete!

[root@03d4589e5dee hello]# nano hello.c

[root@03d4589e5dee hello]# gcc -o hello hello.c -Wall

[root@03d4589e5dee hello]# ls

hello hello.c

[root@03d4589e5dee hello]# ./hello

Hello World!, we are INSO OP from CentOS in a container

[root@03d4589e5dee hello]#

Para terminar la ejecución del contenedor.

```
Installed:
  nano-2.9.8-1.el8.x86_64

Complete!
[root@03d4589e5dee hello]# nano hello.c
[root@03d4589e5dee hello]# gcc -o hello hello.c -Wall
[root@03d4589e5dee hello]# ls
hello  hello.c
[root@03d4589e5dee hello]# ./hello
Hello World!, we are INSO OP from CentOS in a container
[root@03d4589e5dee hello]# exit
```

```
ma@LAPTOP-0VBOP51T: ~
ma@LAPTOP-0VBOP51T:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
15ca772eae9b   centos    "/bin/bash"             8 minutes ago  Up 8 minutes                hungry_wilbur
ma@LAPTOP-0VBOP51T:~$ sudo docker ps
[sudo] password for ma:
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES
ma@LAPTOP-0VBOP51T:~$
```

Para terminar la ejecución de daemond.

```
ma@LAPTOP-0VBOP51T: ~/SSOO/y21/practica1
d4589e5dee42a9f31ca7e0741ee3af8d09727ab50d0ca3a310df69e4bdcff2 module=libcontai
opic=/tasks/delete type="*events.TaskDelete"
WARN[2021-09-26T15:01:11.283398300+02:00] cleaning up after shim disconnected
dee42a9f31ca7e0741ee3af8d09727ab50d0ca3a310df69e4bdcff2 namespace=moby
INFO[2021-09-26T15:01:11.283458100+02:00] cleaning up dead shim
WARN[2021-09-26T15:01:11.292204300+02:00] cleanup warnings time="2021-09-26T15:
fo msg="starting signal loop" namespace=moby pid=1253
^CINFO[2021-09-26T15:43:50.913243800+02:00] Processing signal 'interrupt'
INFO[2021-09-26T15:43:50.915650200+02:00] stopping event stream following grace
<nil>" module=libcontainerd namespace=moby
INFO[2021-09-26T15:43:50.916886200+02:00] Daemon shutdown complete
INFO[2021-09-26T15:43:50.917635100+02:00] stopping healthcheck following gracef
ibcontainerd
INFO[2021-09-26T15:43:50.917657800+02:00] stopping event stream following grace
context canceled" module=libcontainerd namespace=plugins.moby
ma@LAPTOP-0VBOP51T:~/SSOO/y21/practica1$
```